0612. DESENVOLUPAMENT WEB EN ENTORN CLIENT

Curs 2025 - 2026

Objectius

Què farem?

🎯 Aquesta setmana entendràs:

- Fer peticions amb fetch i async/await
- Gestionar errors (HTTP + try/catch)
- Modelar **estats d'UI**: loading / error / empty / ready
- Mantenir arquitectura: **State → Render → Storage**
- Aplicar patrons d'events (inclosa delegació)
- Aplicar CSS accelerat 2/3: espaiat + variants + coherència

Objectius

Connexió amb les sessions anteriors

- Timers (feedback temporal)
- Web Storage (persistència + robustesa)
- Arquitectura base (state/render/storage) + CSS 1/3

Avui: afegim **dades asíncrones** i **UI states** sense trencar el patró.

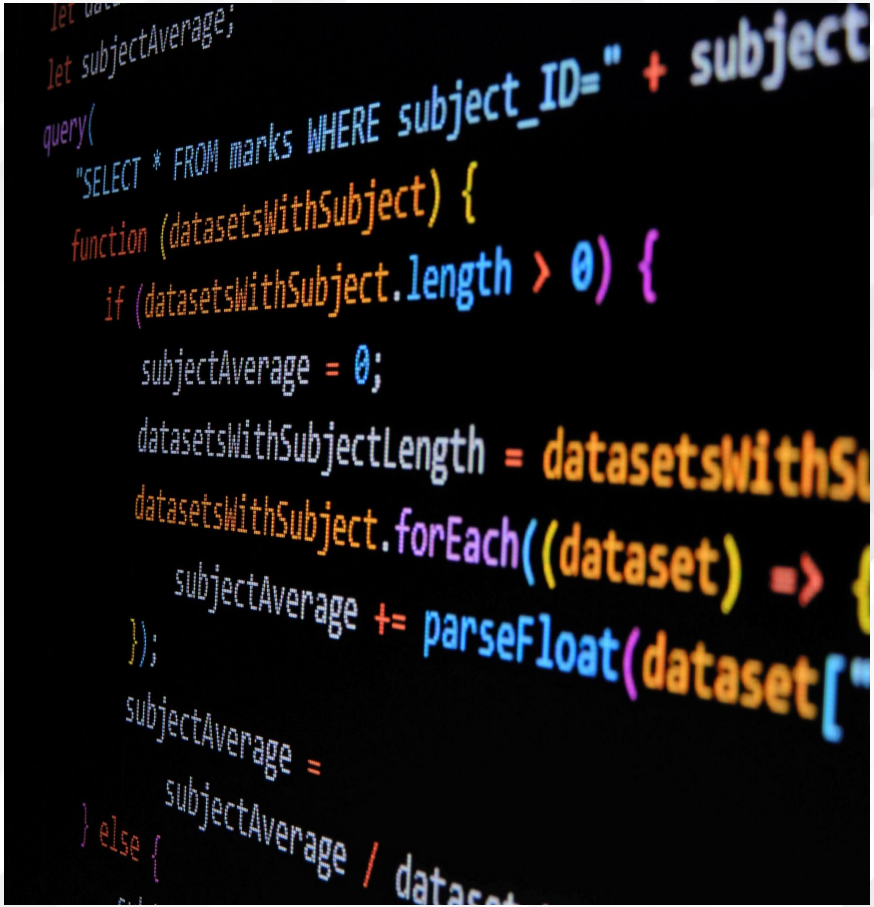
ASINCRONÍA AL CLIENT FETCH

Concepte: I/O asíncron

Quan el navegador fa I/O (llegir dades):

- Pot tardar
- Pot fallar
- Pot tornar dades inesperades

Conclusió: el nostre codi ha de contemplar **temps** i **error**.



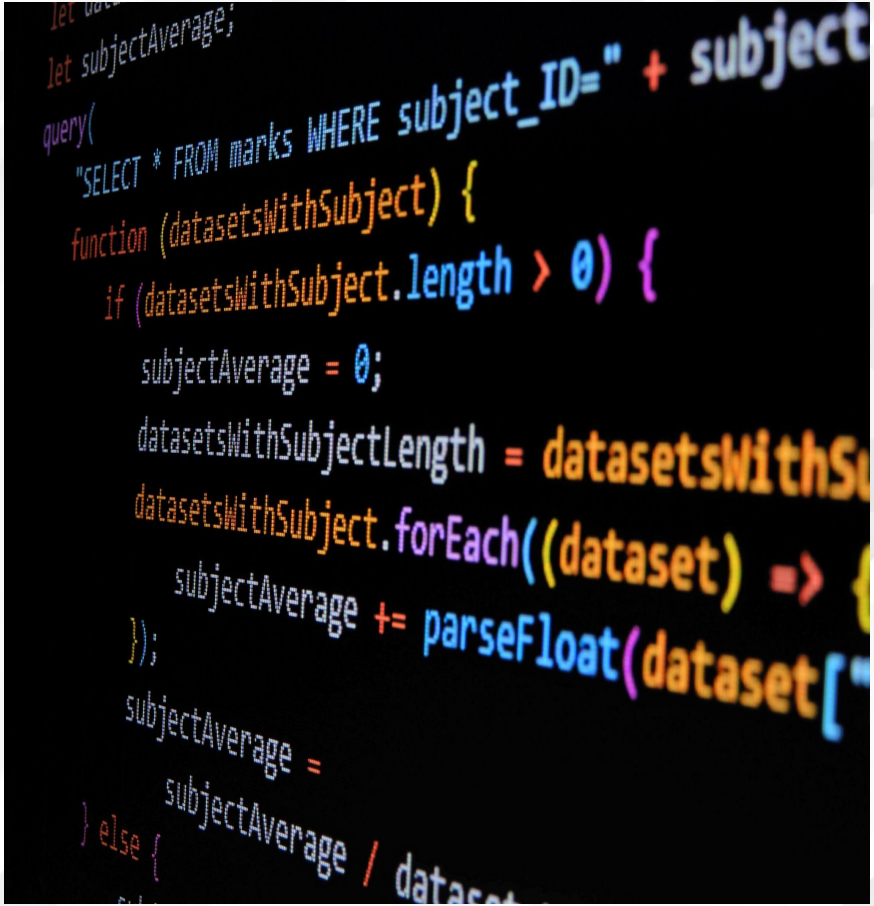
fetch i Promises

fetch(url) retorna una **Promise**:

- “Encara no tinc el resultat”
- Quan arriba → resol amb un Response
- Després parsegem JSON amb response.json()

Idea:

- Treballem amb promeses perquè la web no pot “bloquejar” el fil principal.



Exemple codi: fetch amb async/await

```
async function fetchJson(url) {
    const res = await fetch(url);
    return await res.json();
}
```

Explicació:

- async permet usar await (codi més llegible).
- Aquesta funció encara NO gestiona errors HTTP (404, 500).
- És un primer pas per entendre el flux.

Exemple codi: fetch amb async/await

```
async function fetchJson(url) {
  const response = await fetch(url);

  // Error típic docent: pensar que fetch llença error en 404.
  // No sempre: fetch resol igualment i cal mirar response.ok.
  if (!response.ok) {
    throw new Error(`HTTP ${response.status} - ${response.statusText}`);
  }

  // Aquí pot fallar si el JSON està mal format.
  return await response.json();
}
```

Explicació:

- res.ok evita “parsejar un error com si fos JSON”.
- Convertim l’error HTTP en una excepció (throw) per gestionar-la amb try/catch.
- És un patró professional bàsic.

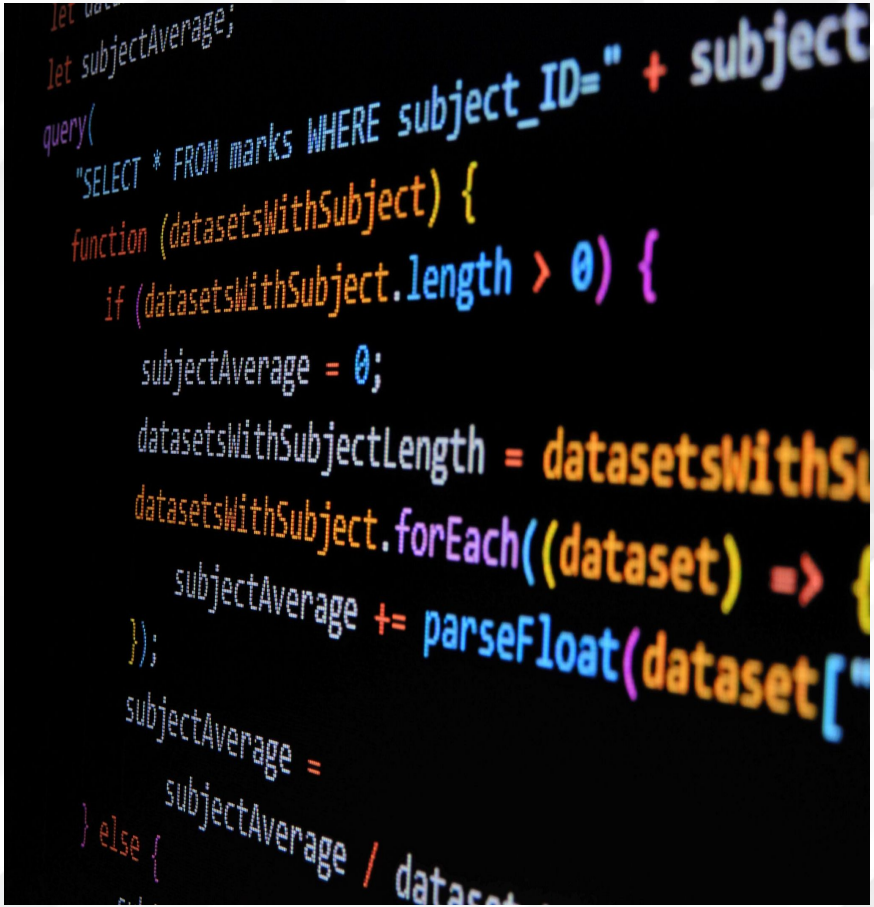
Error típic #1: “si no faig catch, no passa res”

Codi:

```
try {
  const url = scenarioUrl[scenario];
  const data = await fetchJson(url);
```

Concepte:

- Si falla fetchJson, el codi “petarà” i la UI pot quedar buida.
- L’usuari no entén què passa.
- Necessitem UI states + missatge d’error visible.



UI STATES
LA INTERFÍCIE TAMBÉ TÉ ESTAT

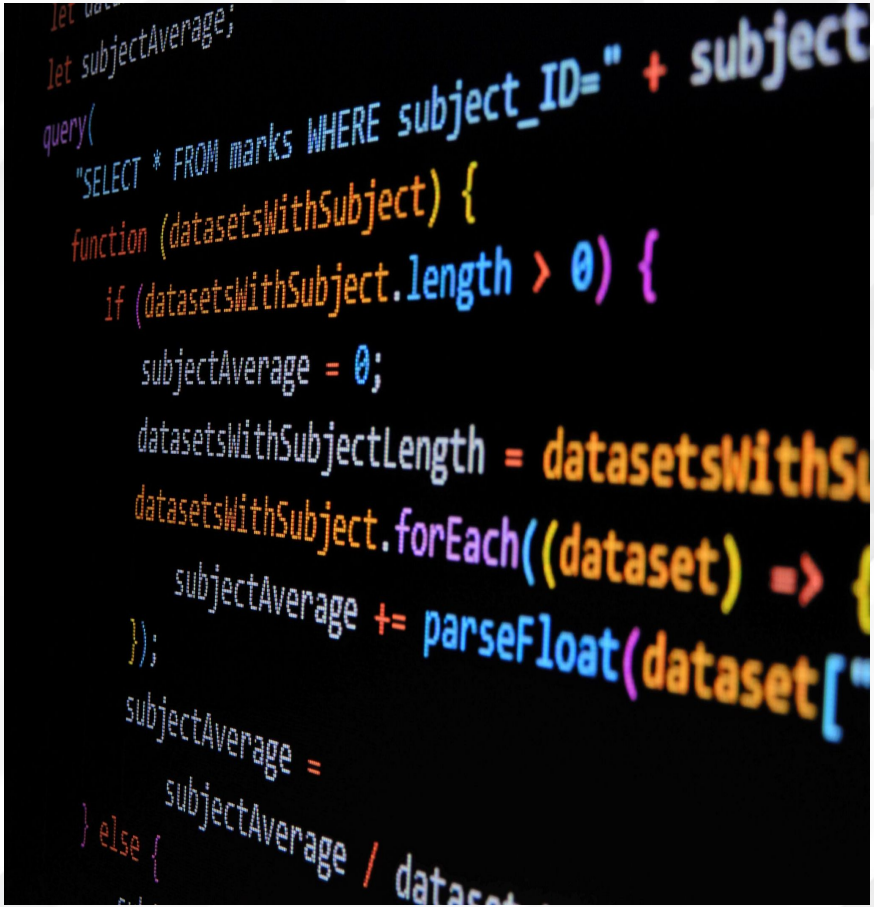
Concepte: status governa la UI

La UI no sempre pot mostrar **“resultats”**:

- Mentre carrega → **loading**
- Si falla → **error**
- Si no hi ha dades → **empty**
- Si tot va bé → **ready**

Concepte

- “Una UI madura sempre sap explicar què està passant.”



Model d'estat mínim (patró reutilitzable)

```
// Model d'estat mínim (patró reusable)
const state = {
  status: "idle",
  // "idle" | "loading" | "ready" | "empty" | "error"
  data: null,
  errorMessage: "",
};
```

Explicació:

- status és una "palanca" per renderitzar pantalles diferents.
- data és el resultat quan està llest.
- errorMessage és per a l'usuari (no només console).

Exemple: flux complet de càrrega amb UI states

Context:

- loading abans de començar.
- ready/empty depèn de la forma de dades (array o null).
- error converteix un problema tècnic en informació útil per UI.

```
async function runScenario(scenario) {
  if (scenario === "idle") {
    setState({ status: "idle", data: null, errorMessage: "" });
    return;
  }

  setState({ status: "loading", data: null, errorMessage: "" });

  try {
    const data = await fetchJson(scenarioUrl[scenario]);

    setState({
      status: isEmpty(data) ? "empty" : "ready",
      data,
      errorMessage: "",
    });
  } catch (error) {
    setState({
      status: "error",
      data: null,
      errorMessage: buildUserMessage(error),
    });

    // Per formació: detall tècnic a DevTools, missatge net a UI.
    console.error("Detall tècnic:", error);
  }
}
```

Exemple: flux complet de càrrega amb UI states

Context:

- loading abans de començar.
- ready/empty depèn de la forma de dades (array o null).
- error converteix un problema tècnic en informació útil per UI.

```
function isEmpty(data) {
  if (Array.isArray(data)) return data.length === 0;
  if (data == null) return true;

  // Si és objecte amb propietat items, també podem considerar empty.
  if (typeof data === "object" && Array.isArray(data.items)) {
    return data.items.length === 0;
  }

  return false;
}
```

Render condicional segons status (snippet)

Context:

- El render és una “centraleta”: segons l’estat, pinta una vista.
- Això evita ifs escampats per tot el projecte.
- Manté l’app coherent quan creix.

```
// Render centralitzat segons status (evita ifs escampats)
function render() {
  badgeEl.textContent = `status: ${state.status}`;
  badgeEl.className = `status-badge ${state.status}`;

  if (state.status === "loading") {
    viewEl.innerHTML = renderLoading();
    return;
  }

  if (state.status === "error") {
    viewEl.innerHTML = renderError(state.errorMessage);
    return;
  }

  if (state.status === "empty") {
    viewEl.innerHTML = renderEmpty();
    return;
  }

  if (state.status === "ready") {
    viewEl.innerHTML = renderReady(state.data);
    return;
  }

  viewEl.innerHTML = renderIdle();
}
```

Error típic #2: UI “en blanc”

Símptoma

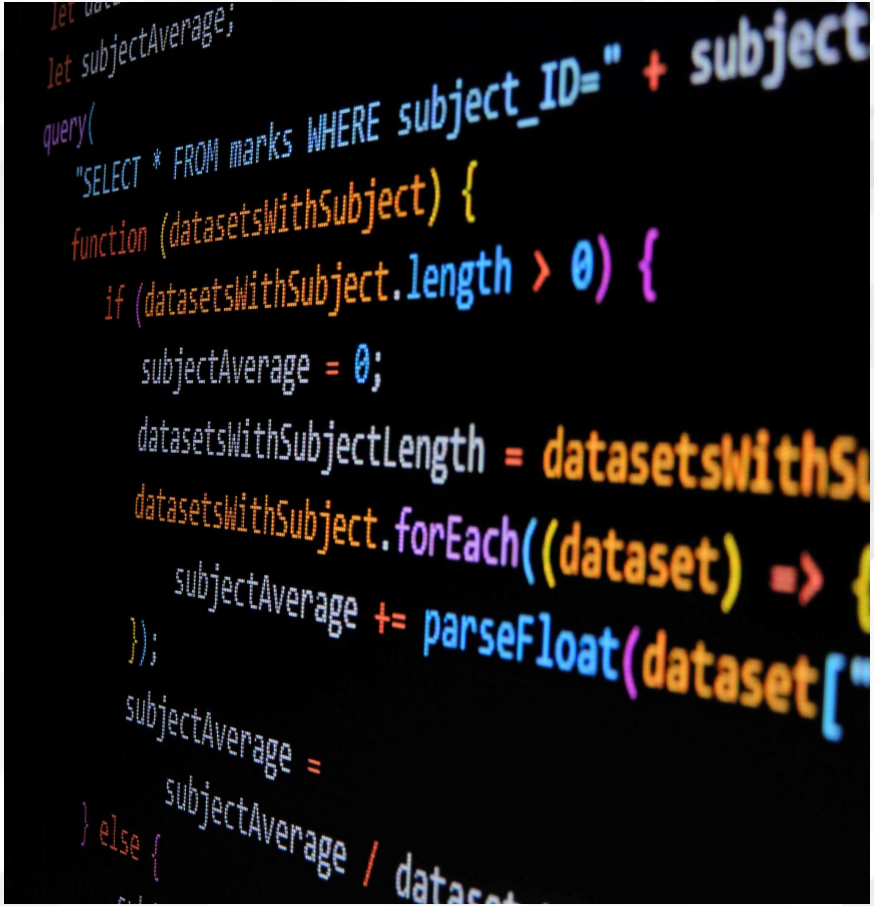
- Carrega… i no es veu res
- O falla… i no es veu res

Causes habituals

- no hi ha status
- no hi ha vista d’error
- render només pensa en “ready”

Solució

- status + renderLoading/renderError/renderEmpty



ARQUITECTURA

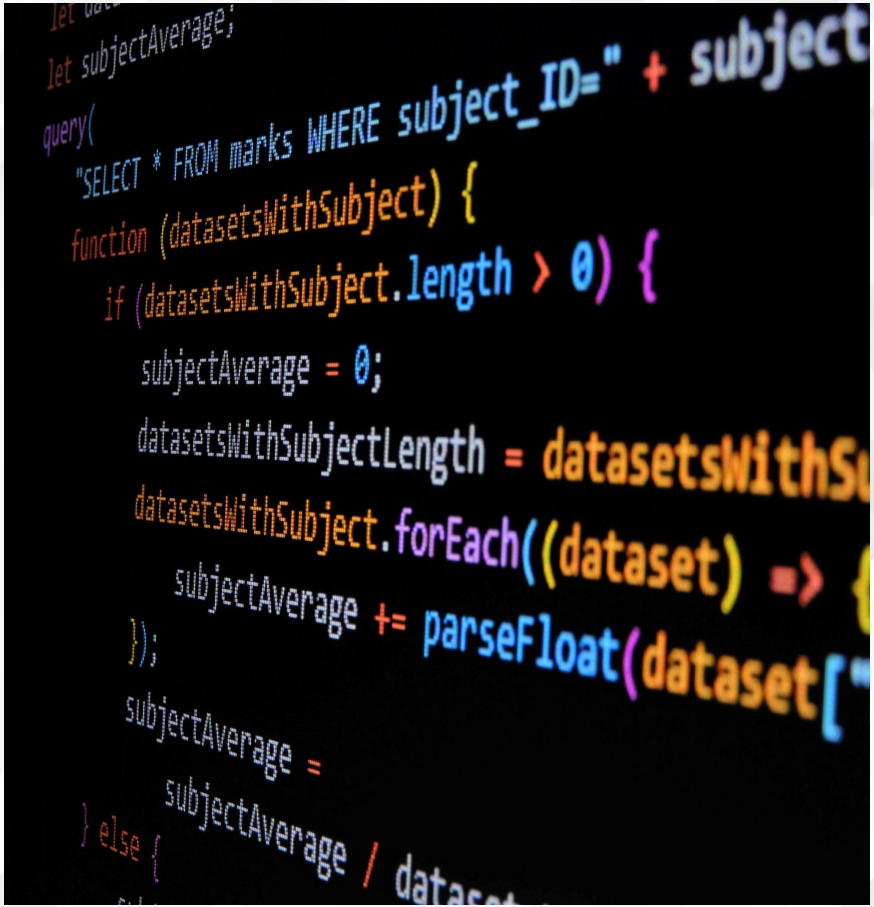
State → Render → Storage

Recordatori: separació de responsabilitats

- **State:** dades (veritat)
- **Render:** pinta
- **Storage:** persisteix
- **Fetch:** alimenta el state (I/O)

Regla d'or

- render() NO llegeix ni escriu storage.



Porta única d’actualització: setState

Explicació

- Canviar estat sempre passa per aquí.
- Evita canvis “fantasma” i facilita depurar.
- Renderitza sempre després de qualsevol canvi.

```
function setState(patch) {
  state = { ...state, ...patch };
  render();
  persistState(patch);
}
```

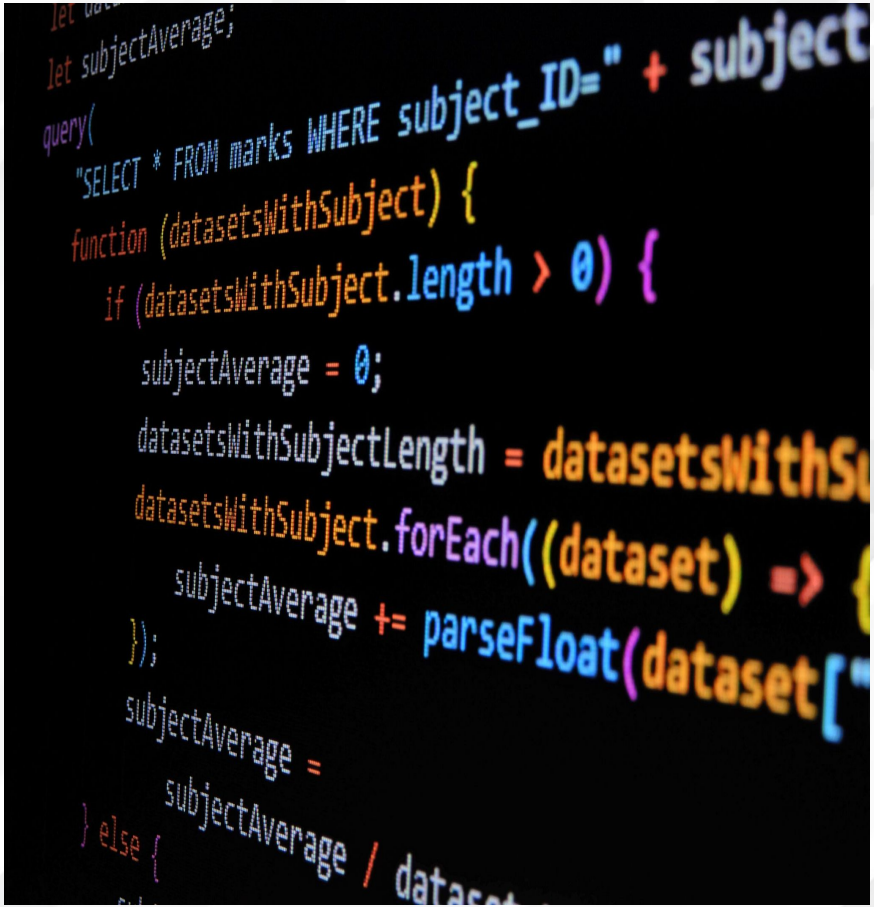
Persistència aplicada

Coses que té sentit persistir (amb keys versionades):

- Preferències (tema, font, motion)
- Últim filtre seleccionat
- Elements “marcats” (IDs: favorits/recents)
- Estat d’interfície reutilitzable (ex: vista triada)

Text per diapo

- Persistir = fer que l’usuari no “comenci de zero” cada vegada.



Exemple: guardar i carregar una llista d'IDs

Explicació:

- Guardar IDs és estable i lleuger.
- null o corrupció → fallback segur.
- Set facilita consultes ràpides.

```
// Per a la llista d'IDs, guardem un array JSON stringificat.
// A la lectura, validem que sigui un array d'enters.
function loadIds() {
  const raw = localStorage.getItem(IDS_KEY);
  if (!raw) return new Set();

  try {
    const arr = JSON.parse(raw);
    return Array.isArray(arr) ? new Set(arr.filter(Number.isFinite)) : new Set();
  } catch {
    return new Set();
  }
}
// Guardem un Set com els IDs favorits, convertit a array per
function saveIds(idsSet) {
  localStorage.setItem(IDS_KEY, JSON.stringify([...idsSet]));
}
```

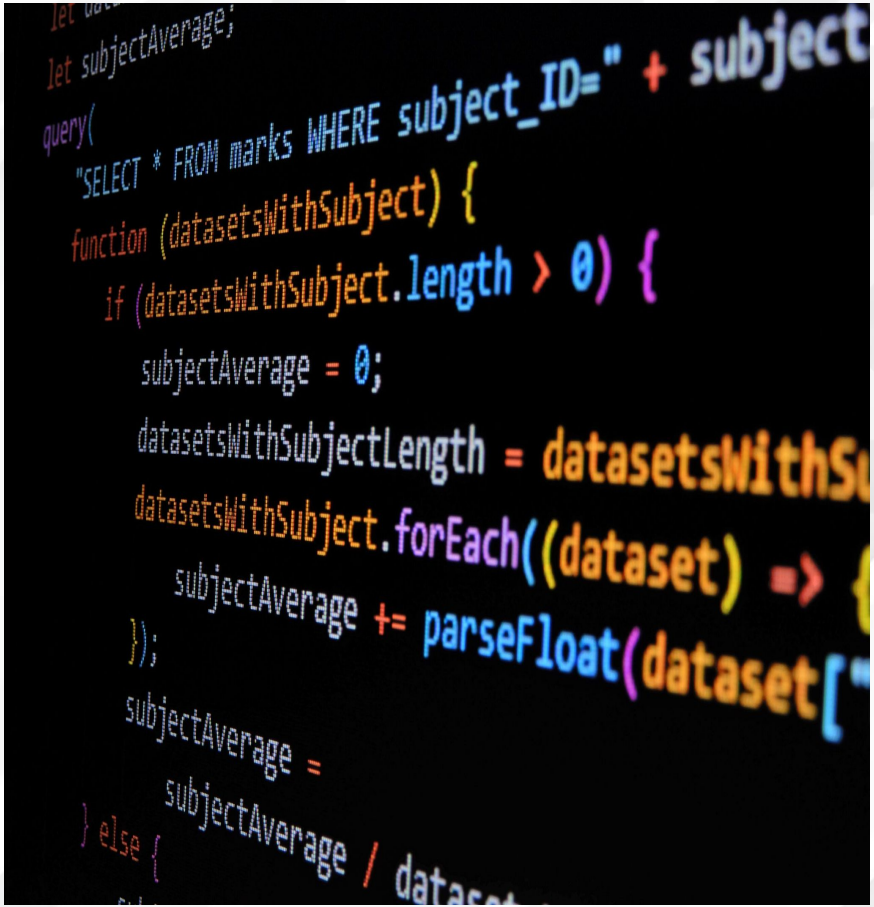
EVENTS
PATRONS I DELEGACIÓ

Concepte: quin event quan?

- input: mentre l'usuari escriu (molt freqüent)
- change: quan "confirma" (més econòmic)
- submit: formulari (evita recarregar)
- click: accions
- focus/blur: validació/UX

Concepte

- "El tipus d'event determina rendiment i experiència."



Porta única d’actualització: setState

Explicació

- change és ideal per persistir preferències sense saturar.
- input pot generar massa escriptures.
- Tria l’event segons el cost del canvi.

```
selectEl.addEventListener("change", (e) => {
  setState({ selected: e.target.value });
  // aquí podries persistir selected, si té sentit
});
```

Delegació d'events

Explicació

- Un sol listener funciona encara que la llista es re-renderitzi.
- data-action permet múltiples accions sense múltiples listeners.
- És el patró recomanat quan generes elements amb render.

```
listEl.addEventListener("click", (e) => {
  const btn = e.target.closest("[data-action]");
  if (!btn) return;

  const item = e.target.closest("[data-id]");
  if (!item) return;

  handleAction(btn.dataset.action, item.dataset.id);
});
```

Error típic #3: listeners “perduts”

Símptoma

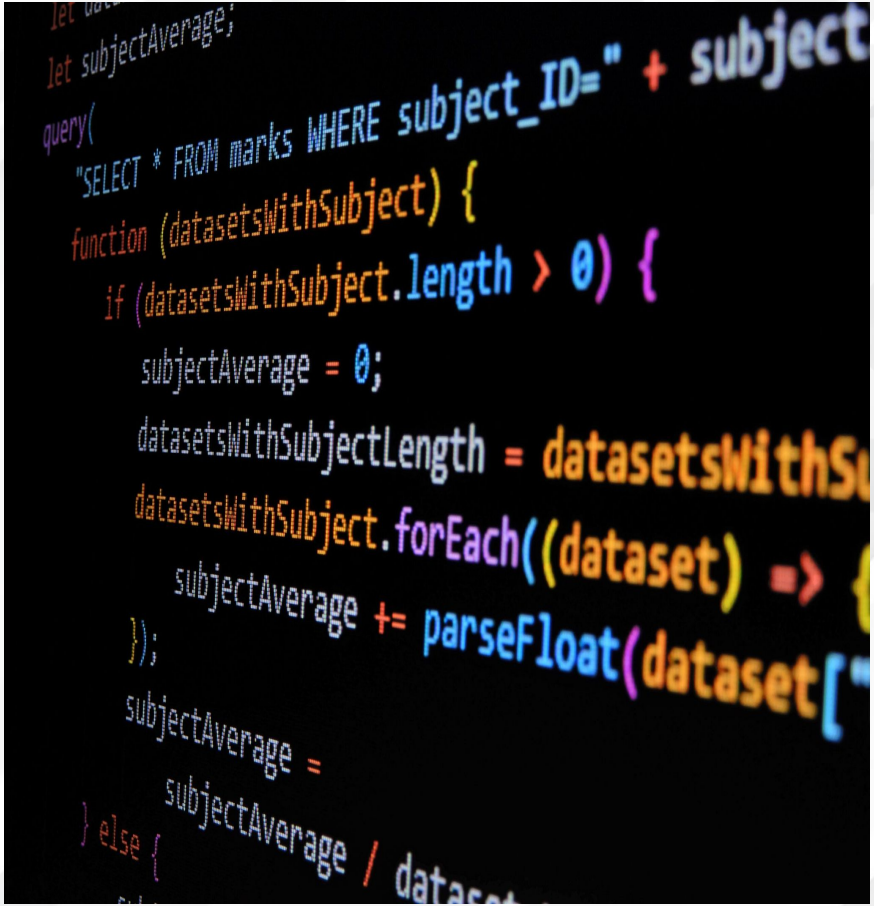
- Renderitzes una llista nova
- Els botons deixen de funcionar

Causa

- Havies afegit listeners a nodes antics
- Els nodes nous no tenen listeners

Solució

- Delegació d’events al contenidor.



FULLES D’ESTIL
CSS
2/3

Objectius

Què farem?

Aquesta setmana afegim:

- Sistema d’espaiat (variables)
- Layout coherent (container + grid/flex)
- Variants de botó (rol visual)
- Responsive mínim (no rebenta)

Nota

- No volem “disseny”, volem **consistència**.

Espaiat com a sistema (snippet)

Explicació:

- Les variables d'espaiat eviten marges aleatoris.
- gap crea separació coherent entre blocs.
- Amb això, el projecte ja es veu "ordenat".

```
:root {
/* Spacing scale */
  --s-1: 8px;   /* Base spacing unit */
  --s-2: 16px;  /* Double the base spacing unit */
  --s-3: 24px;  /* Triple the base spacing unit */
```

```
.container {
  width: min(1000px, 92vw);     /* Max width of 1000px, with 4% horizontal padding on each side */
  margin: 28px auto;    /* Center the container with vertical margin */
  padding: var(--s-3);  /* Add horizontal padding */
  display: grid;    /* Use CSS Grid for layout */
  gap: var(--s-3);  /* Gap between grid items */
}
```

Espaiat com a sistema (snippet)

Explicació:

- Variants indiquen importància o risc (guardar ≠ esborrar).
- Base .btn evita repetir propietats.
- Ajuda a una UX clara.

```
.btn{ padding: 10px 14px; border-radius: 10px; font-weight: 600; }

.btn-primary{ background: var(--accent); color: #fff; border: 0; }
.btn-secondary{ background: transparent; color: var(--text); border: 2px solid var(--border); }
.btn-danger{ background: #dc2626; color: #fff; border: 0; }
```

Focus + responsive

Explicació:

- Focus visible = accessibilitat + qualitat.
- Responsive mínim = l’app no “rebenta” en mòbil.

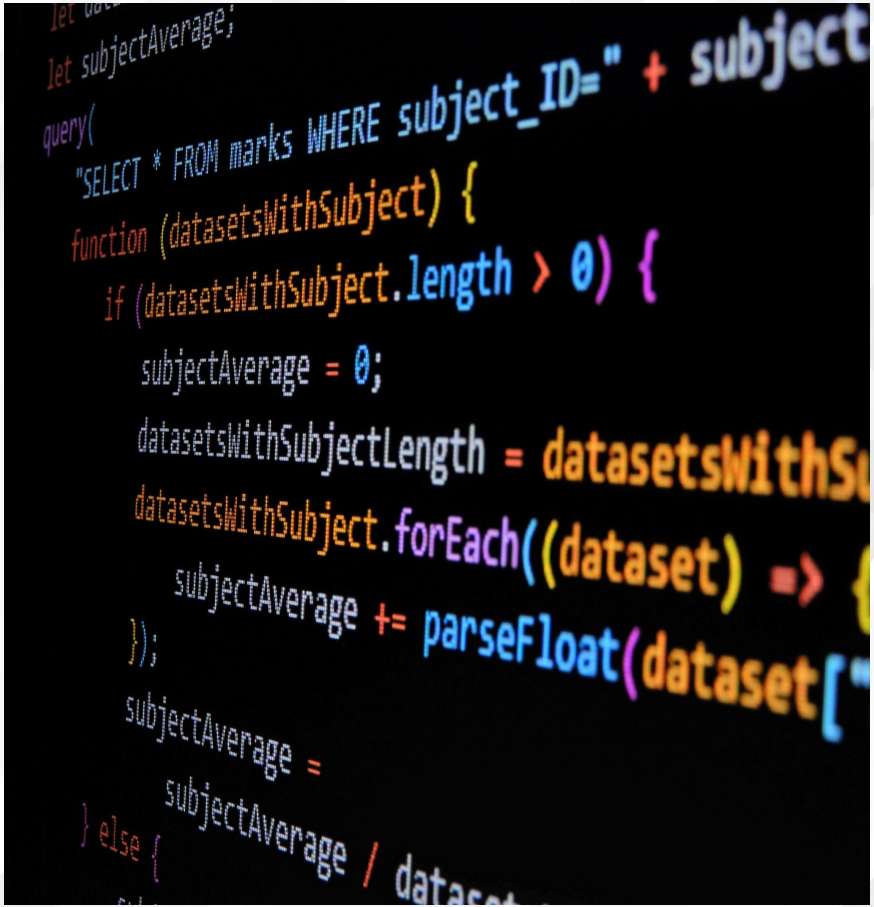
```
.btn:focus, .input:focus{
  outline: none;
  box-shadow: 0 0 0 3px rgba(37,99,235,.35);
}

@media (max-width: 640px){
  .btn{ width: 100%; }
}
```

Overview

Checklist (5 checks)

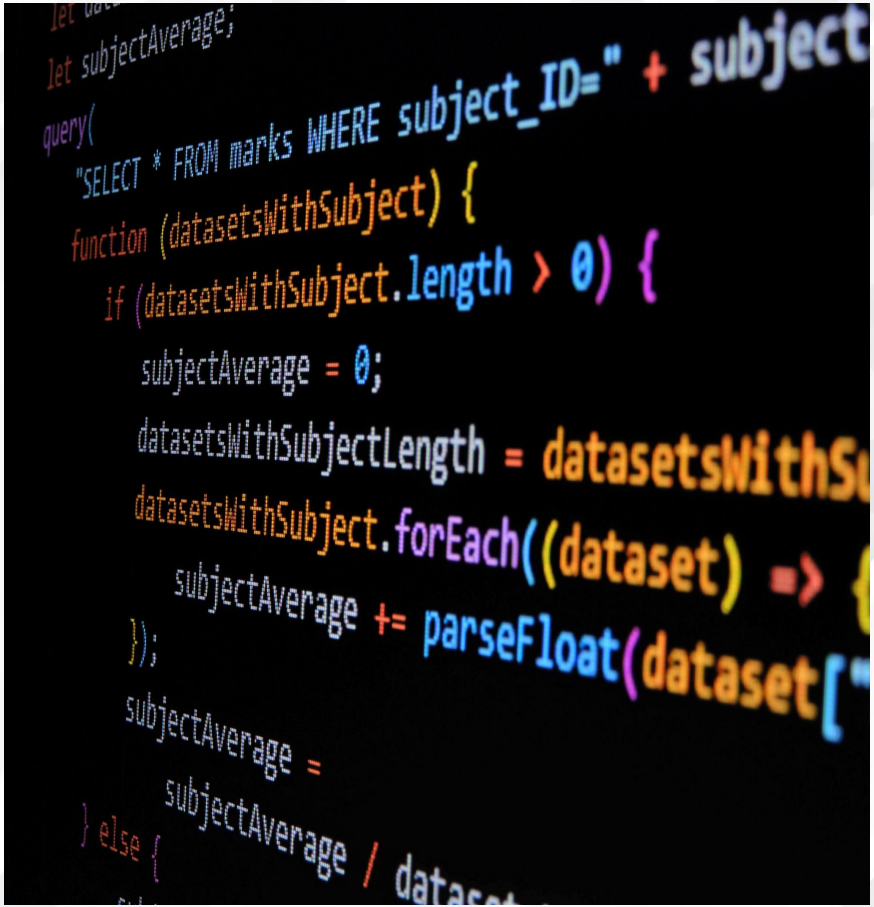
- ✅ Variables d'espaiat --s-1/2/3
- ✅ .container amb gap coherent
- ✅ Variants .btn-primary /.btn-secondary /.btn-danger
- ✅ Focus visible en botons/inputs
- ✅ Responsive mínim per botons o layout



CHECKLIST

Què hem fet avui?

- fetch amb res.ok + try/catch
- state.status controla la UI (loading/error/empty/ready)
- setState(patch) com a porta única
- Persistència robusta (null + corrupció + validació)
- Delegació d'events per llistes dinàmiques
- CSS 2/3 aplicat (espaiat + variants + focus + responsive)



Què ens espera?

Pròxima sessió

- **Continguts**
  - CSS 3/3: motion + reducedMotion + feedback visual d'estat
  - Persistència avançada: mini migració v1 → v2
  - Polir UX: errors i missatges a UI + timers (toast/últim guardat)
- **PE05**

0612. DESENVOLUPAMENT WEB EN ENTORN CLIENT

Curs 2025 - 2026

Contingut elaborat per Martí Zamora

https://creativecommons.org/licenses/by/4.0/